arXiv:2407.12077v1 [cs.CL] 16 Jul 2024

GoldFinch: High Performance RWKV/Transformer Hybrid with Linear Pre-Fill and Extreme KV-Cache Compression

Daniel Goldstein[1,2], **Fares Obeid**[1], **Eric Alcaide**[1,3], **Guangyu Song**[1,4], and **Eugene Cheah**[1,2]

[1]EleutherAI, [2]Recursal AI, [3]Dalle Molle Institute for Artificial Intelligence USI-SUPSI, [4]Tano Labs

Abstract

We introduce GoldFinch, a hybrid Linear Attention/Transformer sequence model that uses a new technique to efficiently generate a highly compressed and reusable KV-Cache in linear time and space with respect to sequence length. GoldFinch stacks our new GOLD transformer on top of an enhanced version of the Finch (RWKV-6) architecture. We train up to 1.5B parameter class models of the Finch, Llama, and GoldFinch architectures, and find dramatically improved modeling performance relative to both Finch and Llama. Our cache size savings increase linearly with model layer count, ranging from 756-2550 times smaller than the traditional transformer cache for common sizes, enabling inference of extremely large context lengths even on limited hardware. Although autoregressive generation has O(n) time complexity per token because of attention, pre-fill computation of the entire initial cache state for a submitted context costs only O(1) time per token due to the use of a recurrent neural network (RNN) to generate this cache. We release our trained weights and training code under the Apache 2.0 license for community use. [1]

1 Introduction

Variations on linear attention (Katharopoulos et al., 2020) have proliferated in recent research (Peng et al., 2024; Qin et al., 2024; Katsch, 2024; Yang et al., 2024), approaching the performance of traditional Multi-Headed Scaled Dot Product Attention (MHA) (Vaswani et al., 2023) while achieving lower inference costs. In MHA, the model's effective memory is bounded by its context length, with the attention calculation resulting in quadratic time complexity with regard to that length. Conversely, most forms of linear attention can be computed recurrently in O(1) time per time-step. Instead of inspecting the entire context length to generate each new token, recurrent linear attention uses a fixed-size hidden state that is updated at each time-step, functioning as its memory of the past. The limited size of this state constrains the capacity of this memory.

The success of Large Language Models (LLMs) has motivated interest in ultra-long context length language models. For example, Gemini Pro (Team et al., 2024) offers a 1 million+ token length window. However, if based on attention, these extra large context lengths come with large associated costs due to the need for MHA to examine every prior token within the context when generating a next token (Liu & Abbeel, 2023; Liu et al., 2023). Although a naive inference implementation would recalculate every key and value at every layer in a traditional transformer, it is common practice to store these in a key-value cache ("KV-Cache")(Pope et al., 2022) and retrieve rather than recompute them. KV-Cache memory costs can be very high. For example, a 1 million token cache for an 80 layer traditional transformer model of hidden dimension 8192 would take up over 2.5 terabytes at bfloat16 precision. We turn our focus to reducing the memory costs of this cache while also reducing computational complexity and memory usage for processing the initial context of a request.

Our contribution is the combination of several innovations to create the GoldFinch architecture, which improves pre-fill and decoding efficiency, as well as downstream modeling performance, and introduces the following innovations:

[1]Code at: https://github.com/recursal/GoldFinch-paper
Model weights at: https://huggingface.co/recursal/GoldFinch-paper

1. employs a novel parameter-efficient modification of Finch (RWKV-6), which we call "Finch-C2", for the first 2/3 of its layers
2. uses these the output of these Finch-C2 layers to produce an extremely small *compressed* global key cache using a novel mechanism we call "TokenCat". Our cache thus requires only $\frac{1}{16}d_{model}$ per token plus the original input token indices, instead of $2d_{model}n_{layer}$ for traditional KV-caches.
3. employs a novel modification of the traditional transformer architecture, which we call "GOLD", for the last 1/3 of its layers to consume this key cache and produce outputs without even requiring a traditional value cache.

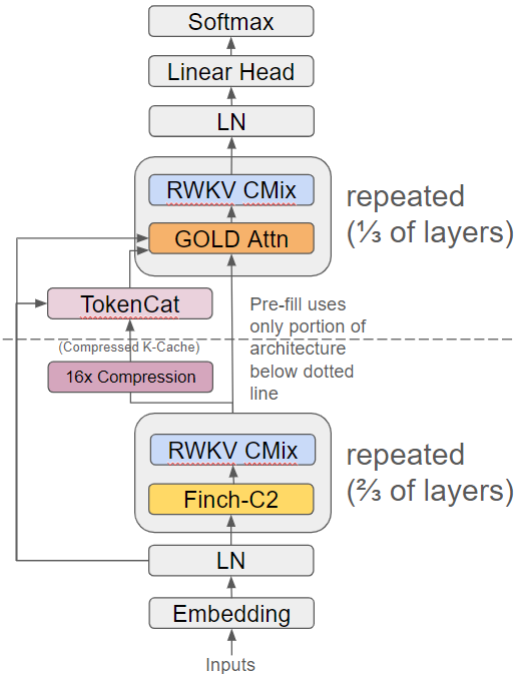

Figure 1: GoldFinch Architecture Block Diagram

Architecture	**Pre-fill time complexity per token**	**KV-Cache entries per token**	**KV-Cache Bytes 256k context, 32 layers 4096 hidden dim**
Llama2	$O(N)$	$2d_{model}n_{layer}$	128GB
Llama3 (w/ GQA)	$O(N)$	$8d_{head}n_{layer}$	32GB
DeepSeek-V2	$O(N)$	$\frac{9}{2}d_{head}n_{layer}$	18GB
Zamba	$O(N)$	$\frac{2}{7}d_{model}n_{layer}$	18.3GB
Jamba	$O(N)$	$\frac{8}{7}d_{head}n_{layer}$	4GB
YOCO	$\mathbf{O(1)}$	$2d_{model}$	4GB
GoldFinch	$\mathbf{O(1)}$	$\mathbf{1+\frac{d_{model}}{16}}$	**0.068GB**

Table 1: Time and space complexity comparisons of models with full softmax attention. No KV-Cache quantization is shown.

The GOLD layers are an adaptation of a novel improved transformer we call "GPTAlpha" that can also be used as a standalone transformer model for improved non-hybrid performance.

This new architecture brings a series of significant benefits:

1. We are able to reuse the same KV-Cache on every transformer layer while maintaining greater than Llama (Touvron et al., 2023) performance. This reduces the KV-Cache size by a factor of the total number of layers of the model.

2. We eliminate the values from the KV-Cache, leaving only a key cache. Instead of caching values, we store the input indices and generate the values from these, reducing the KV-Cache size by another factor of nearly 2.
3. We are able to compress our key cache by applying a form of Low Rank Adaptation (LoRA) (Hu et al., 2021) to the output of a single layer, and re-expanding the compressed version by concatenating the compressed version with the original token embeddings, further reducing the size by 128 times. ("TokenCat")
4. We use the input embedding table and RWKV-style token shift to generate values for attention without sacrificing performance.
5. By using Finch-C2 blocks at the start of the model, the key cache automatically encodes the underlying implicit positional representation, thereby removing the need for positional encoding within our transformer layers for trained context lengths. We do still require an additional positional encoding method for extrapolation to new context lengths unseen during training.
6. There are many use cases of LLMs that involve relatively short responses to questions about long documents. Because our compressed key cache is generated by an RNN with an operating time and space complexity of O(1) per token with regard to sequence length, we are able to generate the cache in these cases extremely inexpensively and apply the O(N) per token cost GOLD transformer portion of our calculations only to new token generation, for which relatively few iterations are often required.

To obtain our Finch-C2 architecture we improve the Finch time-mixer by removing the gate, swapping out GroupNorm for a LayerNorm across all heads, doing a new multiplication (of the key by one minus the decay) to keep the kv-state rows normalized, and replacing Finch's u ("bonus") term with a new data-dependent separately token-shifted second Value. These changes result in improved performance with little to no speed penalty and significantly fewer total parameters.

To obtain our GPTAlpha architecture we improve the Llama architecture by replacing the transformer feed-forward network (FFN) with the RWKV channel mixer, and adding RWKV style token shifts and extra LayerNorms to attention layers.

Both Finch-C2 and GPTAlpha can be used either as standalone model architectures with improved performance over their counterparts, or as part of the GoldFinch hybrid model architecture.

The GOLD transformer architecture (GPTAlpha Over Linear transformer Decoder) removes the key and value weights from GPTAlpha in favor of producing keys and values from a combination of the original token indices passed through the embeddings table, a highly compressed version of the outputs of the Finch-C2 layers, and a data-driven LoRA.

GoldFinch stacks a set of GOLD transformer layers on top of a Finch-C2 linear transformer, passing the outputs for the Finch layers both into a key compressor to be stored for every sequence position, and also through the current timestep as part of the normal residual stream.

We train GoldFinch models up to 1.45 billion parameters on 1.5 trillion tokens of *minipile* (Kaddour, 2023) and compare them to slightly larger equivalently trained Finch (Peng et al., 2024) and Llama (Touvron et al., 2023) models. We find that GoldFinch significantly outperforms both Llama and Finch in downstream performance and perplexity across nearly every benchmark we tested, while maintaining fewer parameters, a much smaller cache than Llama, and perfect MQAR recall due to its use of full attention.

2 Background

Transformers have become the de-facto choice for most sequence modeling tasks, and have been shown to be especially effective in the context of language modeling. However, they present computational challenges when processing long context lengths, which has hindered their adoption for long sequence tasks. Specifically, the formulation of multi-head scaled dot-product attention (MHA) has a computational complexity of O(N^2) with respect to context length. Additionally, inference engines typically rely on the use of a KV-Cache to enable autoregressive token generation in O(N) time per token. This cache grows linearly with context length, and becomes challenging to fit into limited Video Random-Access Memory (VRAM) for longer sequences.

Recent transformer models such as the Llama series rely on Grouped-Query Attention (GQA) (Ainslie et al., 2023) to help ameliorate this cache size problem. At a typical number of groups $n_g = 8$, GQA reduces the KV-Cache size by $\frac{n_g}{n_h}$ times, where n_h is the number of heads. This is helpful, especially on consumer grade hardware, but leads to a reduction in downstream performance, and longer sequences still cause a significant problem in terms of VRAM usage.

The recently proposed YOCO (Sun et al., 2024) improves the computational complexity for pre-fill of the initial request context and also reduces the KV-cache size by introducing a new global KV-Cache instead of the usual per-layer cache. The computational improvement is achieved by replacing the first half of the layers in the model with Linear Attention based RetNet-G layers (Sun et al., 2023), which is a recurrent neural network (RNN) architecture that requires only linear time with respect to sequence length. YOCO stores the output of these first layers as a global KV-Cache, which is then used by the second half of the layers, featuring MHA. Overall, this reduces the KV-Cache size by a factor of the number of layers, without a reported performance reduction. Goldfinch takes a related approachetNet-G, and processes the output differently, creating an effective but much smaller cache via our TokenCat mechanism, which is then consumed by our enhanced transformer GOLD layers.

Hungry Hungry Hippos (H3) (Fu et al., 2023) train a hybrid recurrent SSM/transformer model containing just two layers of attention, which they find outperforms transformers. This served as a warning shot that SSM(or linear attention)-transformer hybrids have the potential to step in as higher performance replacements for transformers alone.

Recognizing the challenges posed at inference time by the KV-Cache, DeepSeek-V2 (DeepSeek-AI et al., 2024) proposes a replacement for MHA called Multi-head Latent Attention (MLA). This uses low-rank joint key-value compression to reduce the size of the KV-Cache from $2n_h d_h l$ to $\frac{9}{2} d_h l$, equivalent to the KV-Cache size required for GQA with only 2.25 groups. Because the low-rank key-value compression requires fewer parameters than full rank key and value matrices, MLA achieves greater per-parameter performance than MHA. GoldFinch also improves performance via this kind of compression-based relative parameter reduction.

HGRN2 (Qin et al., 2024) replaces the per-head GroupNorm (Wu & He, 2018) with a full-width LayerNorm, and we do the same in our Finch-C2 architecture. HGRN2 sets their key to be equal to one minus the decay, and we do something related but slightly different, multiplying our key by one minus the decay.

Inspired by these works, we propose a new method that further reduces the KV-Cache by orders of magnitude and reduces the cost of the initial context load to become linear with respect to sequence length, all while achieving greater than Llama performance.

2.1 Other Concurrent Related Work

Other concurrent work on hybrid models bear some similarities to portions of our architecture:

Zamba (Glorioso et al., 2024) interleaves Global Shared Attention (GSA) every N Mamba blocks (Gu & Dao, 2024). Instead of using the residual output of the prior Mamba block as its input, Zamba concatenates the original embeddings generated before layer zero onto this residual output, and use the double-width combination as the input to attention. Although their GSA blocks share parameters, they are not able to share the same KV-Cache. The concatenation of embeddings bears similarity to our new "TokenCat" technique.

Jamba (Lieber et al., 2024) is a mixture-of-experts (MoE) (Shazeer et al., 2017) Mamba-based (Gu & Dao, 2024) model that inserts attention layers periodically within its architecture, for a total of 1:7 ratio of attention-to-Mamba layers. Similarly to Goldfinch's ability to rely upon RWKV's implicit positional encoding within the pre-trained context length, they find that explicit positional encoding may not be required for their hybrid Mamba-based architecture.

Samba (Ren et al., 2024) is a hybrid model that repeats blocks containing a Mamba layer, an MLP layer, a sliding-window attention (SWA) layer featuring RoPE (Su et al., 2023), and another MLP layer. The use of SWA allows a fixed cost of execution per token, regardless of context length.

3 Method

GoldFinch follows the general structure of the Finch architecture, which is also the common pre-norm decoder transformer structure used in Llama and RWKV. It consists of a series of layers, each containing a time mixing sub-layer followed by a channel mixing sub-layer. All channel mixing sub-layers are Finch channel mixers.

The following formulae describe the three varieties of GoldFinch sub-layers. All matrices $\boldsymbol{W}$ are learned per layer, unless described otherwise. We show all time mixing formulae per-head for conciseness, except the formulae for those layer outputs where heads are combined via $concat$. Model dimension is denoted as D, head size as H, and number of heads as N. All values are $\in \mathbb{R}^H$ unless otherwise noted.

3.1 Finch-C2 Time Mixing

The first two-thirds of time mixing sub-layers use a variation on the Finch time mixer we call Finch-C2.

We customize the Finch time-mixing sub-layers by removing the gate, swapping out GroupNorm for a LayerNorm across all heads and doing a new multiplication of the key by one minus the decay. Finally, we replace Finch's u ("bonus") term with a new data-dependent separately token-shifted second Value, computed using the same weights as the base Value, with an additional LoRA added to the result. We find that this allows us to remove all of the Gate parameters while retaining performance.

Along the lines of (Peng et al., 2024), we introduce the following notation for common operators in the model, using the square subscript to denote a variable:

$$\text{lerp}(a, b, t) = a + (b - a) \odot t, \tag{1}$$

$$\text{lora}_{\square}(x) = \lambda_{\square} + \tanh(x\boldsymbol{A}_{\square})\boldsymbol{B}_{\square}, \tag{2}$$

$$\text{ddlerp}_{\square}(a, b) = a + (b - a) \odot \text{lora}_{\square}(a + (b - a) \odot \mu_x), \tag{3}$$

Then, the Finch-C2 block can be formalized as:

$$d_t = \text{lora}_!(\text{ddlerp}_d(x_t, x_{t-1})), \tag{4}$$
$$w_t = \exp(-\exp(d_t)), \tag{5}$$
$$r_t = \text{ddlerp}_r(x_t, x_{t-1})\boldsymbol{W}^R, \tag{6}$$
$$k_t = \text{ddlerp}_k(x_t, x_{t-1})\boldsymbol{W}^K \cdot (1 - w_t), \tag{7}$$
$$v_t = \text{ddlerp}_v(x_t, x_{t-1})\boldsymbol{W}^V, \tag{8}$$
$$u_t = \text{ddlerp}_u(x_t, x_{t-1}), \tag{9}$$
$$u'_t = u_t\boldsymbol{W}^V + \tanh(u_t\boldsymbol{W}^{UD})\boldsymbol{W}^{UU}. \tag{10}$$
$$\tag{11}$$

And after splitting the hidden dimension into N heads:

$$\boldsymbol{wkv}_t = \sum_{i=1}^{t-1} \text{diag}\left(\bigodot_{j=i+1}^{t-1} w_j\right) \cdot k_i^{\mathrm{T}} \cdot v_i \in \mathbb{R}^{H \times H}, \tag{12}$$

$$o_t = \text{LayerNorm}(\text{concat}\left(r_t \cdot \boldsymbol{wkv}_t + u'_t\right))\boldsymbol{W}^O \in \mathbb{R}^D. \tag{13}$$

Please note that the calculation for u'_t reuses the same weights $\boldsymbol{W}^V$ - this is an intentional parameter count savings and not a typo.

3.2 GOLD Key Compression

The output from the first two-thirds of the model is used in two ways: it is passed on to the next layer in the usual manner, and also compressed down via multiplication with the global (not per-layer) learned matrix $\boldsymbol{W}^{KD} \in \mathbb{R}^{Dx(D/16)}$ to one sixteenth its original size and stored into a unified single-layer compressed key cache:

$$c_t = x_t \boldsymbol{W}^{KD} \in \mathbb{R}^{(D/16)}. \tag{14}$$

3.3 GOLD Key Decompression (TokenCat)

The compressed key cache is decompressed via a two-step method. The first step is "TokenCat", short for "Token conCatenation", in which the compressed key is concatenated with the original input token embedding from the very beginning of the model. The concatenated result is then multiplied with the global (not per-layer) learned matrix $\boldsymbol{W}^{KU} \in \mathbb{R}^{(D+D/16)xD}$ and RMSNormed to obtain the decompressed attention proto-keys, which are common to all GOLD attention sub-layers.

$$k_t^D = \text{RMSNorm}\left(\text{concat}\left(x_t^0, c_t\right) \boldsymbol{W}^{KU}\right). \tag{15}$$

3.4 GOLD Attention Time Mixing

The remaining time mixing sub-layers are a variation on GPTAlpha attention sub-layers employing MHA that we call GOLD attention.

Each GOLD attention sub-layer calculates its own unique attention keys and values from the decompressed proto-keys and the original input token embeddings, respectively. Each is passed through a data-dependent token shift, with the result passed through an additive LoRA. We call this process "DDLoRAdapt", introducing the relevant notation below, using the square subscript to denote a variable:

$$\text{loradapt}_{\square}(x) = x + \tanh(x\boldsymbol{C}_{\square})\boldsymbol{D}_{\square}. \tag{16}$$

The following are the formulae for GOLD attention time mixing:

$$q_t = \text{LayerNorm}(\text{ddlerp}_q(x_t, x_{t-1})\boldsymbol{W}^Q), \tag{17}$$

$$a_t = \text{lerp}(x_t^0, x_{t-1}^0, \mu_x), \tag{18}$$

$$k_t = \text{LayerNorm}\left(\text{loradapt}_k\left(\text{lerp}\left(k_t^D, k_{t-1}^D, \text{lora}_k\left(a_t\right)\right)\right)\right), \tag{19}$$

$$v_t = \text{LayerNorm}\left(\text{loradapt}_v\left(\text{lerp}\left(x_t^0, x_{t-1}^0, \text{lora}_v\left(a_t\right)\right)\right)\right), \tag{20}$$

$$o_t = \text{LayerNorm}(\text{concat}\left(\text{attention}(q_t, k, v)\right)\boldsymbol{W}^O \in \mathbb{R}^D. \tag{21}$$

Please note the receptance-like Finch style token-shift on queries, and additional data-driven token-shift on keys and values, with keys being reconstituted from compressed key cache entries c_t and values coming from the original token embeddings x^0. x^0 is the embedding input to the first sub-layer in the model, and can be reconstituted during inference from the token indices by storing those indices, usually only an additional two bytes per context length.

Data dependent token shift (ddlerp) is a specialized low-parameter cost variety of two-step 1D convolution that originated in the RWKV architecture. It allows the model to dynamically linearly interpolate between the current and previous time-step on a per channel basis. We use our DDLoRAdapt version of the technique to inexpensively apply contextual information to the keys and values, increasing the amount of information from which they are generated without significantly increasing parameter count.

Note that the token shift cannot be dependent on the hidden-state, as that would make recurrent calculation impossible for older keys and values, and would require a full KV-Cache to be stored. Instead, we use the original input token embeddings as the data upon which the key and value token-shifts depend.

Pre-fill of the compressed key cache to prepare for autoregressive generation can be computed in linear time with respect to the number of tokens. This is accomplished by running only the Finch-C2 section of the model on those tokens. One important implementation caveat is that token shift requires the prior layer hidden-state output from the previous time-step. At first glance this appears problematic, as the GOLD layers require full quadratic attention, which is what we were trying to avoid during pre-fill. But the solution is simple: given G GOLD layers in the model, there must be $2G-1$ sub-layers that require such a previous time-step hidden state but are directly or indirectly reliant on the outputs of quadratic attention. Therefore, the last $2G-1$ tokens of pre-fill must be run through the full model (not just the Finch-C2 layers) to generate these hidden-states. These $2G-1$ computations can be done in a single call to the full model to leverage the same kinds of parallelism used during training.

Only the compressed key cache entries and original input token indices must be permanently kept in VRAM during inference, as the key cache can be reconstituted via decompression on-demand.

Because decompression and token shift can be done on contiguous regions of key value pairs instead of all of them at once, extremely low VRAM usage can be achieved during inference by calculating attention incrementally across the sequence for each layer and decompressing as you go.

3.5 GoldFinch Channel Mixing (same as Finch Channel Mixing)

Goldfinch channel mixing is identical to Finch channel mixing. It is used as the feed forward network component on all layers of the model, both Finch-C2 and GOLD. We reproduce it here for reference. Please note that variables have their own independent definitions in this subsection.

$$r_t = \text{lerp}_r(x_t, x_{t-1}, \mu_r)\boldsymbol{W}^R \in \mathbb{R}^D, \tag{22}$$

$$k_t = \text{lerp}_k(x_t, x_{t-1}, \mu_k)\boldsymbol{W}^K \in \mathbb{R}^{3.5D}, \tag{23}$$

$$v_t = \text{ReLU}(k_t)^2\boldsymbol{W}^V \in \mathbb{R}^D, \tag{24}$$

$$o_t = \sigma(r_t) \odot v_t \in \mathbb{R}^D. \tag{25}$$

3.6 GPTAlpha Time Mixing

For completeness and to show how it can be used in a pure transformer architecture, we list the formulae for GPTAlpha time mixing when not used in conjunction with TokenCat below:

$$q_t = \text{LayerNorm}(\text{ddlerp}_q(x_t, x_{t-1})\boldsymbol{W}^Q), \tag{26}$$

$$k_t = \text{LayerNorm}(\text{ddlerp}_k(x_t, x_{t-1})\boldsymbol{W}^K), \tag{27}$$

$$v_t = \text{LayerNorm}(\text{ddlerp}_v(x_t, x_{t-1})\boldsymbol{W}^V), \tag{28}$$

$$o_t = \text{LayerNorm}(\text{concat}\big(\text{attention}(q_t, k, v)\big))\boldsymbol{W}^O \in \mathbb{R}^D. \tag{29}$$

4 Experiments

4.1 Architecture Comparisons

We trained 1.5B parameter-class models with 24 layers, 2048 hidden-dimension, 2048 context length of Finch, Llama, and GoldFinch for comparison on *minipile* (Kaddour, 2023), all using the same RWKV World tokenizer. GoldFinch ends with dramatically lower final loss than the others (by over 0.1 out of 2.39), and uses over 100 million fewer parameters than its Finch counterpart.

We additionally trained a GoldFinch with no compression, to show that there is very little lost with our choice of a 16:1 hidden-dimension compression ratio.

In the interest of fairly comparing performance for Llama by giving it the most favorable conditions, we add the RWKV small init embeddings optimization (LayerNorm after embeddings with small initialized values) (Peng et al., 2023) and do not employ Grouped Query Attention. All architectures used the same hyperparameters and were trained on 4 GPUs, with per-GPU per-step batch size of 8, two steps of gradient accumulation, and a 10 step learning rate warm-up followed by cosine decay annealed from 3e-5 to 1e-5. We train with Adam betas of 0.9 and 0.99, epsilon 1e-8 and weight decay 0.001. Weight decay was applied only to matrix parameters that are not part of LoRAs or the GoldFinch key compression/expansion steps.

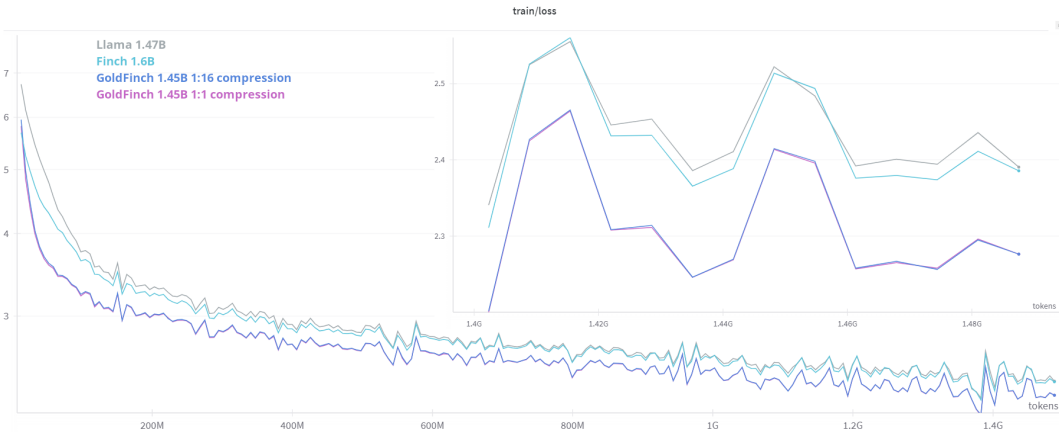

Figure 2: Loss curves of 1.5B class models.

Architecture (L24 D2048 ctx2048)	**Parameters**	**Loss** ↓
Llama	1.47B	2.3905
Finch	1.60B	2.3856
GoldFinch, last 1/3 layers GOLD, 16:1 compression	1.45B	**2.2762**
GoldFinch, last 1/3 layers GOLD, 1:1 compression	1.45B	**2.2762**

Table 2: Final loss values for various models of size L24 D2048 ctx2048 trained on *minipile*

In addition to comparing training and validation losses, we ran a series of common benchmark evaluations on the three 1.5B parameter class models trained on *minipile*. Finch and Llama scored similarly to one another, and GoldFinch significantly outperformed both.

Model	**lmbd ppl** ↓	**avg acc** ↑	**lmbd acc** ↑	**piqa acc** ↑	**hella acc** ↑	**winog acc** ↑	**arc_c acc** ↑	**arc_e acc** ↑	**sciq acc** ↑
Finch 1.60B	81.9	42.8%	24.3%	62.4%	28.7%	49.0%	**19.6%**	44.9%	70.8%
Llama 1.47B	71.7	43.0%	26.3%	61.6%	28.1%	50.5%	19.3%	43.9%	71.0%
GoldFinch 1.45B	**48.2**	**44.2%**	**29.1%**	**63.4%**	**29.1%**	**50.2%**	18.3%	**45.9%**	**73.7%**

Table 3: Common benchmark evaluations for various models of size L24 D2048 ctx2048 trained on *minipile*

4.2 Ablation Studies

We ran various smaller scale ablation studies to determine the contributions of different parts of the GoldFinch architecture relative to both Finch, Llama, GPTAlpha, and a hybrid of our improved Finch and GPTAlpha with no KV-Cache compression or key/value sharing. The new second value added in Finch-C2 had the smallest positive impact of anything measured. Surprisingly,

GoldFinch performed very slightly better than even the Finch-C2/GPTAlpha hybrid with no KV compression at all. Each test trained a 12 layer 768 hidden-dimension model at 1024 context length with the same RWKV World tokenizer on the full *minipile* dataset. All architectures used the same hyperparameters and were trained on single GPUs, with per-step batch size of 32, two steps of gradient accumulation, and a 10 step learning rate warm-up followed by cosine decay annealed from 6e-5 to 2e-5. We train with Adam betas of 0.9 and 0.99, epsilon 1e-8 and weight decay 0.001. Weight decay was applied only to matrix parameters that are not part of LoRAs or the GoldFinch key compression/expansion steps.

Architecture (L12 D768 ctx1024)	**Loss** ↓
Finch-C2 without $k* = 1 - w$	2.7293
Finch	2.7191
Llama	2.7125
Finch-C2 without second value	2.7105
Finch-C2	2.7082
GPTAlpha with RoPE	2.6684
GoldFinch, last 1/2 layers GOLD	2.6637
GoldFinch, last 1/3 layers GOLD with RoPE	2.6590
Finch-C2, last 1/3 layers GPTAlpha	2.6586
GoldFinch, last 1/3 layers GOLD	2.6582
GoldFinch, last 1/6 layers GOLD	**2.6578**

Table 4: Final loss values for various ablations of model size L12 D768 ctx1024 trained on *minipile*

4.3 Associative Recall

Associative recall (AR) is a synthetic task designed to emulate the human ability to associate and retrieve information. It evaluates a model's skill in recalling previously mentioned information within a given context. Previous studies suggest that a model's performance in AR is a good indicator of its efficacy in in-context learning (Elhage et al., 2021; Olsson et al., 2022). Consequently, AR has been employed as a benchmark for developing new language model architectures (Fu et al., 2023; Poli et al., 2023; Lutati et al., 2023). Arora et al. (2023) evaluated a variety of models for multi-query associative recall (MQAR) and discovered a performance gap between different linear transformer architectures and the traditional transformer with attention.

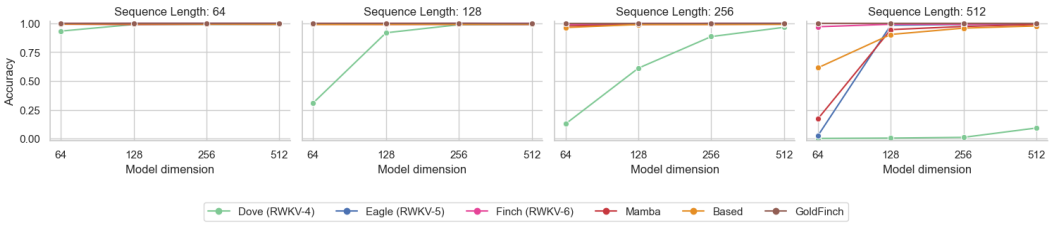

Figure 3: MQAR tasks. An increase in sequence length correlates with increased task difficulty.

In Figure 3, we used the same experimental settings as Arora et al. (2023) and show that GoldFinch achieves perfect MQAR scores, outperforming traditional attention-free language models. As a hybrid architecture that leverages attention, GoldFinch can solve MQAR as well as transformer models with attention. Additionally, we trained GoldFinch on a context length of 1024 to demonstrate that this trend continues, as depicted in Figure 4.

4.4 Long Context Experiments

We tested the loss of our small Finch and GoldFinch models pre-trained on *minipile* at all context lengths up to 65536 on the *PG19* (Rae et al., 2019) dataset of older books. These pre-trained models were all trained at only 1024 context length. The Finch model is able to maintain a fairly low loss throughout the 65536 context length. The base GoldFinch model trained with no positional

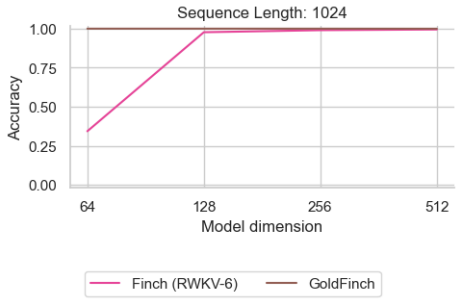

Figure 4: Finch and GoldFinch on the same MQAR task with increased sequence length

encoding goes up in loss significantly starting at around double the trained context length, then plateauing at a high loss. The GoldFinch model trained with RoPE on its GOLD attention sub-layers performs better, but loss still increases somewhat as the sequence progresses. However, by applying interpolated RoPE values we are able to obtain low loss throughout the extended context length. We conclude that for GoldFinch models in which extrapolation beyond the maximum trained context length is desired, the GOLD attention sub-layers should be trained with RoPE, with interpolation employed upon inference.

We then fine-tuned the RoPE and non-RoPE models mentioned above on 165 million tokens of *minipile* at longer context lengths. During this fine-tuning, we froze the entire RWKV portion of the model up to the first GOLD layer, allowing the optimizer to update the parameters of only the GOLD layers and output head. This saves a significant amount of time and VRAM during fine-tuning, allowing an even longer context length to fit into memory and using roughly 3x fewer FLOPS per token. We theorize that because the GOLD attention portion of the model can use keys generated from the RWKV output, this is enough to support sophisticated attention matching across the entire context length.

Our experiments showed that indeed the RoPE model with GOLD layers fine-tuned at longer context lengths exhibited significantly lower losses against *PG19* up through those lengths and even beyond. On the non-RoPE model this process was somewhat successful within the fine-tuned context length, while still failing at extrapolation. This was unexpected, since the RWKV layers were not updated and the GOLD layers included no positional encoding mechanism. We postulate that token-shift may supply some minimal positional information to the model.

4.5 Checkpoint Upgrade Training

We have attempted upgrading existing pre-trained Finch models to a more limited version of GoldFinch that uses the Finch architecture for its RWKV layers instead of the Finch-C2 component. We tried many variations on two methods, one that adds new GOLD layers on top for a total of around 11% more parameters, and another which keeps the layer count the same as the pre-trained model. Thus far with only small amounts of upgrade training neither method has performed to our satisfaction.

Both methods were attempted on a 1.6B Finch checkpoint that had been pre-trained on 2.5 trillion tokens.

For the first method we appended 4 GOLD layers on top of the pre-trained 1.6B Finch checkpoint before the language modeling head, and continued training it for 100 million tokens using two different learning rates. The original 24 pre-trained layers were kept at the same 1e-5 LR at which their pre-training had ended upon, while the LR for the 4 new GOLD layers was annealed along a cosine schedule from 3e-4 to 1e-5. While the performance of this model was in line with the original model, it was unclear if the resultant model from this method really learned anything of value in its GOLD layers.

The second method involved freezing the embedding and RWKV layers and importing but not freezing the final 1/3 of the channel mixer sub-layers that were paired with freshly initialized GOLD attention sub-layers. We then trained this model on a relatively small amount of data

(in our case around 7.5 billion tokens of a new internal dataset) while annealing the learning rate to the final learning rate seen in the pre-trained base model. The resultant model obtained a similar validation loss on *minipile* to the base model, despite being trained on a completely different dataset and the base model having been already trained for over 2.25 trillion tokens. However, the new model's LAMBADA scores were worse. We attribute this loss of performance to the 'brain surgery' required to keep the layer count the same, in which we effectively erased the Finch time-mix parameters in the upper 1/3rd of the model.

We are still doing further experimentation on these upgrade methods to see just how well they can be made to perform. We hope to be able to inexpensively upgrade even the largest 14B Finch model to this reduced GoldFinch format and see significant performance improvements at larger context lengths due to the GOLD attention being able to look back across the entire context with no state-size based memory limitations.

5 Further Work

We anticipate updating this pre-print with further studies as results become available, including checkpoint upgrade results and evaluations, longer experiment training runs, and new long context experiments. Please check back for updates.

Most of the experiments done for this pre-print were performed over a short period of time on a single node containing 8 RTX 4090 cards. In the future we hope to demonstrate GoldFinch's performance on larger models with significantly more tokens.

We expect that GoldFinch will work similarly with other linear attention and SSM architectures in place of the Finch-C2 blocks. For example, it should be possible to implement a "GoldMamba" architecture in the same style.

Further work might explore increased memory reduction for the global KV-Cache via quantization, and application of ring attention Liu et al. (2023) to lower the memory requirements when extending to very long contexts. As a hybrid architecture model, GoldFinch will likely benefit from any future improvements to the RWKV and transformer architectures.

6 Conclusion

We have introduced a hybrid RNN-Attention model architecture (GoldFinch) and trained models that demonstrate its performance up to 1.45B. The resulting hybrid RNN-Attention models combine the efficiency of RNNs with the capabilities of attention-based models. Having RNNs for the initial layers allows for fast pre-fill and removes the need for positional encoding on the RNN layers, while the attention layers improve associative recall. The combination with a highly compressed global KV-Cache unlocks a memory reduction in inference while maintaining enhanced performance. We release the trained weights and training code under the Apache 2.0 license.

7 Acknowledgements

Special thanks to Bo Peng for his tireless dedication to the RWKV architecture and community. The main GoldFinch code herein was based on a modified version of his public Linear Attention Arena code repository, and upgraded models were based on his pre-trained Finch model releases.

References

Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. Gqa: Training generalized multi-query transformer models from multi-head checkpoints, 2023.

Simran Arora, Sabri Eyuboglu, Aman Timalsina, Isys Johnson, Michael Poli, James Zou, Atri Rudra, and Christopher Re. Zoology: Measuring and improving recall in efficient language models, 2023.

DeepSeek-AI, Aixin Liu, Bei Feng, Bin Wang, Bingxuan Wang, Bo Liu, Chenggang Zhao, Chengqi Dengr, Chong Ruan, Damai Dai, Daya Guo, Dejian Yang, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Hanwei Xu, Hao Yang, Haowei Zhang, Honghui Ding, Huajian Xin, Huazuo Gao, Hui Li, Hui Qu, J. L. Cai, Jian Liang, Jianzhong Guo, Jiaqi Ni, Jiashi Li, Jin Chen, Jingyang Yuan, Junjie Qiu, Junxiao Song, Kai Dong, Kaige Gao, Kang Guan, Lean Wang, Lecong Zhang, Lei Xu, Leyi Xia, Liang Zhao, Liyue Zhang, Meng Li, Miaojun Wang, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Mingming Li, Ning Tian, Panpan Huang, Peiyi Wang, Peng Zhang, Qihao Zhu, Qinyu Chen, Qiushi Du, R. J. Chen, R. L. Jin, Ruiqi Ge, Ruizhe Pan, Runxin Xu, Ruyi Chen, S. S. Li, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shaoqing Wu, Shengfeng Ye, Shirong Ma, Shiyu Wang, Shuang Zhou, Shuiping Yu, Shunfeng Zhou, Size Zheng, T. Wang, Tian Pei, Tian Yuan, Tianyu Sun, W. L. Xiao, Wangding Zeng, Wei An, Wen Liu, Wenfeng Liang, Wenjun Gao, Wentao Zhang, X. Q. Li, Xiangyue Jin, Xianzu Wang, Xiao Bi, Xiaodong Liu, Xiaohan Wang, Xiaojin Shen, Xiaokang Chen, Xiaosha Chen, Xiaotao Nie, Xiaowen Sun, Xiaoxiang Wang, Xin Liu, Xin Xie, Xingkai Yu, Xinnan Song, Xinyi Zhou, Xinyu Yang, Xuan Lu, Xuecheng Su, Y. Wu, Y. K. Li, Y. X. Wei, Y. X. Zhu, Yanhong Xu, Yanping Huang, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Li, Yaohui Wang, Yi Zheng, Yichao Zhang, Yiliang Xiong, Yilong Zhao, Ying He, Ying Tang, Yishi Piao, Yixin Dong, Yixuan Tan, Yiyuan Liu, Yongji Wang, Yongqiang Guo, Yuchen Zhu, Yuduan Wang, Yuheng Zou, Yukun Zha, Yunxian Ma, Yuting Yan, Yuxiang You, Yuxuan Liu, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhen Huang, Zhen Zhang, Zhenda Xie, Zhewen Hao, Zhihong Shao, Zhiniu Wen, Zhipeng Xu, Zhongyu Zhang, Zhuoshu Li, Zihan Wang, Zihui Gu, Zilin Li, and Ziwei Xie. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model, 2024.

Nelson Elhage, Neel Nanda, Catherine Olsson, Tom Henighan, Nicholas Joseph, Ben Mann, Amanda Askell, Yuntao Bai, Anna Chen, Tom Conerly, Nova DasSarma, Dawn Drain, Deep Ganguli, Zac Hatfield-Dodds, Danny Hernandez, Andy Jones, Jackson Kernion, Liane Lovitt, Kamal Ndousse, Dario Amodei, Tom Brown, Jack Clark, Jared Kaplan, Sam McCandlish, and Chris Olah. A mathematical framework for transformer circuits. *Transformer Circuits Thread*, 2021. https://transformer-circuits.pub/2021/framework/index.html.

Daniel Y. Fu, Tri Dao, Khaled K. Saab, Armin W. Thomas, Atri Rudra, and Christopher Ré. Hungry hungry hippos: Towards language modeling with state space models, 2023.

Paolo Glorioso, Quentin Anthony, Yury Tokpanov, James Whittington, Jonathan Pilault, Adam Ibrahim, and Beren Millidge. Zamba: A compact 7b ssm hybrid model, 2024.

Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces, 2024.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models, 2021.

Jean Kaddour. The minipile challenge for data-efficient language models, 2023.

Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers are rnns: Fast autoregressive transformers with linear attention, 2020.

Tobias Katsch. Gateloop: Fully data-controlled linear recurrence for sequence modeling, 2024.

Opher Lieber, Barak Lenz, Hofit Bata, Gal Cohen, Jhonathan Osin, Itay Dalmedigos, Erez Safahi, Shaked Meirom, Yonatan Belinkov, Shai Shalev-Shwartz, Omri Abend, Raz Alon, Tomer Asida, Amir Bergman, Roman Glozman, Michael Gokhman, Avashalom Manevich, Nir Ratner, Noam Rozen, Erez Shwartz, Mor Zusman, and Yoav Shoham. Jamba: A hybrid transformer-mamba language model, 2024.

Hao Liu and Pieter Abbeel. Blockwise parallel transformers for large context models. In *Neural Information Processing Systems*, 2023. URL https://api.semanticscholar.org/CorpusID:266351737.

Hao Liu, Matei Zaharia, and Pieter Abbeel. Ring attention with blockwise transformers for near-infinite context, 2023.

Shahar Lutati, Itamar Zimerman, and Lior Wolf. Focus your attention (with adaptive iir filters), 2023.

Catherine Olsson, Nelson Elhage, Neel Nanda, Nicholas Joseph, Nova DasSarma, Tom Henighan, Ben Mann, Amanda Askell, Yuntao Bai, Anna Chen, Tom Conerly, Dawn Drain, Deep Ganguli, Zac Hatfield-Dodds, Danny Hernandez, Scott Johnston, Andy Jones, Jackson Kernion, Liane Lovitt, Kamal Ndousse, Dario Amodei, Tom Brown, Jack Clark, Jared Kaplan, Sam McCandlish, and Chris Olah. In-context learning and induction heads, 2022.

Bo Peng, Eric Alcaide, Quentin Anthony, Alon Albalak, Samuel Arcadinho, Stella Biderman, Huanqi Cao, Xin Cheng, Michael Chung, Leon Derczynski, Xingjian Du, Matteo Grella, Kranthi Gv, Xuzheng He, Haowen Hou, Przemyslaw Kazienko, Jan Kocon, Jiaming Kong, Bartłomiej Koptyra, Hayden Lau, Jiaju Lin, Krishna Sri Ipsit Mantri, Ferdinand Mom, Atsushi Saito, Guangyu Song, Xiangru Tang, Johan Wind, Stanisław Woźniak, Zhenyuan Zhang, Qinghua Zhou, Jian Zhu, and Rui-Jie Zhu. RWKV: Reinventing RNNs for the transformer era. In Houda Bouamor, Juan Pino, and Kalika Bali (eds.), *Findings of the Association for Computational Linguistics: EMNLP 2023*, pp. 14048–14077, Singapore, December 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.findings-emnlp.936. URL https://aclanthology.org/2023.findings-emnlp.936.

Bo Peng, Daniel Goldstein, Quentin Anthony, Alon Albalak, Eric Alcaide, Stella Biderman, Eugene Cheah, Xingjian Du, Teddy Ferdinan, Haowen Hou, Przemysław Kazienko, Kranthi Kiran GV, Jan Kocoń, Bartłomiej Koptyra, Satyapriya Krishna, Ronald McClelland Jr. au2, Niklas Muennighoff, Fares Obeid, Atsushi Saito, Guangyu Song, Haoqin Tu, Stanisław Woźniak, Ruichong Zhang, Bingchen Zhao, Qihang Zhao, Peng Zhou, Jian Zhu, and Rui-Jie Zhu. Eagle and finch: Rwkv with matrix-valued states and dynamic recurrence, 2024.

Michael Poli, Stefano Massaroli, Eric Nguyen, Daniel Y Fu, Tri Dao, Stephen Baccus, Yoshua Bengio, Stefano Ermon, and Christopher Ré. Hyena hierarchy: Towards larger convolutional language models. In *International Conference on Machine Learning*, pp. 28043–28078. PMLR, 2023.

Reiner Pope, Sholto Douglas, Aakanksha Chowdhery, Jacob Devlin, James Bradbury, Anselm Levskaya, Jonathan Heek, Kefan Xiao, Shivani Agrawal, and Jeff Dean. Efficiently scaling transformer inference. *ArXiv*, abs/2211.05102, 2022. URL https://api.semanticscholar.org/CorpusID:253420623.

Zhen Qin, Songlin Yang, Weixuan Sun, Xuyang Shen, Dong Li, Weigao Sun, and Yiran Zhong. Hgrn2: Gated linear rnns with state expansion, 2024.

Jack W Rae, Anna Potapenko, Siddhant M Jayakumar, Chloe Hillier, and Timothy P Lillicrap. Compressive transformers for long-range sequence modelling. *arXiv preprint*, 2019. URL https://arxiv.org/abs/1911.05507.

Liliang Ren, Yang Liu, Yadong Lu, Yelong Shen, Chen Liang, and Weizhu Chen. Samba: Simple hybrid state space models for efficient unlimited context language modeling, 2024.

Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer, 2017.

Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding, 2023.

Yutao Sun, Li Dong, Shaohan Huang, Shuming Ma, Yuqing Xia, Jilong Xue, Jianyong Wang, and Furu Wei. Retentive network: A successor to transformer for large language models, 2023.

Yutao Sun, Li Dong, Yi Zhu, Shaohan Huang, Wenhui Wang, Shuming Ma, Quanlu Zhang, Jianyong Wang, and Furu Wei. You only cache once: Decoder-decoder architectures for language models, 2024.

Gemini Team, Machel Reid, Nikolay Savinov, Denis Teplyashin, Dmitry, Lepikhin, Timothy Lillicrap, Jean baptiste Alayrac, Radu Soricut, Angeliki Lazaridou, Orhan Firat, Julian Schrittwieser, Ioannis Antonoglou, Rohan Anil, Sebastian Borgeaud, Andrew Dai, Katie Millican, Ethan Dyer, Mia Glaese, Thibault Sottiaux, Benjamin Lee, Fabio Viola, Malcolm Reynolds, Yuanzhong Xu, James Molloy, Jilin Chen, Michael Isard, Paul Barham, Tom Hennigan, Ross McIlroy, Melvin Johnson, Johan Schalkwyk, Eli Collins, Eliza Rutherford, Erica Moreira, Kareem Ayoub, Megha Goel, Clemens Meyer, Gregory Thornton, Zhen Yang, Henryk Michalewski, Zaheer Abbas, Nathan Schucher, Ankesh Anand, Richard Ives, James Keeling, Karel Lenc, Salem Haykal, Siamak Shakeri, Pranav Shyam, Aakanksha Chowdhery, Roman Ring, Stephen Spencer, Eren Sezener, Luke Vilnis, Oscar Chang, Nobuyuki Morioka, George Tucker, Ce Zheng, Oliver Woodman, Nithya Attaluri, Tomas Kocisky, Evgenii Eltyshev, Xi Chen, Timothy Chung, Vittorio Selo, Siddhartha Brahma, Petko Georgiev, Ambrose Slone, Zhenkai Zhu, James Lottes, Siyuan Qiao, Ben Caine, Sebastian Riedel, Alex Tomala, Martin Chadwick, Juliette Love, Peter Choy, Sid Mittal, Neil Houlsby, Yunhao Tang, Matthew Lamm, Libin Bai, Qiao Zhang, Luheng He, Yong Cheng, Peter Humphreys, Yujia Li, Sergey Brin, Albin Cassirer, Yingjie Miao, Lukas Zilka, Taylor Tobin, Kelvin Xu, Lev Proleev, Daniel Sohn, Alberto Magni, Lisa Anne Hendricks, Isabel Gao, Santiago Ontanon, Oskar Bunyan, Nathan Byrd, Abhanshu Sharma, Biao Zhang, Mario Pinto, Rishika Sinha, Harsh Mehta, Dawei Jia, Sergi Caelles, Albert Webson, Alex Morris, Becca Roelofs, Yifan Ding, Robin Strudel, Xuehan Xiong, Marvin Ritter, Mostafa Dehghani, Rahma Chaabouni, Abhijit Karmarkar, Guangda Lai, Fabian Mentzer, Bibo Xu, YaGuang Li, Yujing Zhang, Tom Le Paine, Alex Goldin, Behnam Neyshabur, Kate Baumli, Anselm Levskaya, Michael Laskin, Wenhao Jia, Jack W. Rae, Kefan Xiao, Antoine He, Skye Giordano, Lakshman Yagati, Jean-Baptiste Lespiau, Paul Natsev, Sanjay Ganapathy, Fangyu Liu, Danilo Martins, Nanxin Chen, Yunhan Xu, Megan Barnes, Rhys May, Arpi Vezer, Junhyuk Oh, Ken Franko, Sophie Bridgers, Ruizhe Zhao, Boxi Wu, Basil Mustafa, Sean Sechrist, Emilio Parisotto, Thanumalayan Sankaranarayana Pillai, Chris Larkin, Chenjie Gu, Christina Sorokin, Maxim Krikun, Alexey Guseynov, Jessica Landon, Romina Datta, Alexander Pritzel, Phoebe Thacker, Fan Yang, Kevin Hui, Anja Hauth, Chih-Kuan Yeh, David Barker, Justin Mao-Jones, Sophia Austin, Hannah Sheahan, Parker Schuh, James Svensson, Rohan Jain, Vinay Ramasesh, Anton Briukhov, Da-Woon Chung, Tamara von Glehn, Christina Butterfield, Priya Jhakra, Matthew Wiethoff, Justin Frye, Jordan Grimstad, Beer Changpinyo, Charline Le Lan, Anna Bortsova, Yonghui Wu, Paul Voigtlaender, Tara Sainath, Shane Gu, Charlotte Smith, Will Hawkins, Kris Cao, James Besley, Srivatsan Srinivasan, Mark Omernick, Colin Gaffney, Gabriela Surita, Ryan Burnell, Bogdan Damoc, Junwhan Ahn, Andrew Brock, Mantas Pajarskas, Anastasia Petrushkina, Seb Noury, Lorenzo Blanco, Kevin Swersky, Arun Ahuja, Thi Avrahami, Vedant Misra, Raoul de Liedekerke, Mariko Iinuma, Alex Polozov, Sarah York, George van den Driessche, Paul Michel, Justin Chiu, Rory Blevins, Zach Gleicher, Adrià Recasens, Alban Rrustemi, Elena Gribovskaya, Aurko Roy, Wiktor Gworek, Sébastien M. R. Arnold, Lisa Lee, James Lee-Thorp, Marcello Maggioni, Enrique Piqueras, Kartikeya Badola, Sharad Vikram, Lucas Gonzalez, Anirudh Baddepudi, Evan Senter, Jacob Devlin, James Qin, Michael Azzam, Maja Trebacz, Martin Polacek, Kashyap Krishnakumar, Shuo yiin Chang, Matthew Tung, Ivo Penchev, Rishabh Joshi, Kate Olszewska, Carrie Muir, Mateo Wirth, Ale Jakse Hartman, Josh Newlan, Sheleem Kashem, Vijay Bolina, Elahe Dabir, Joost van Amersfoort, Zafarali Ahmed, James Cobon-Kerr, Aishwarya Kamath, Arnar Mar Hrafnkelsson, Le Hou, Ian Mackinnon, Alexandre Frechette, Eric Noland, Xiance Si, Emanuel Taropa, Dong Li, Phil Crone, Anmol Gulati, Sébastien Cevey, Jonas Adler, Ada Ma, David Silver, Simon Tokumine, Richard Powell, Stephan Lee, Kiran Vodrahalli, Samer Hassan, Diana Mincu, Antoine Yang, Nir Levine, Jenny Brennan, Mingqiu Wang, Sarah Hodkinson, Jeffrey Zhao, Josh Lipschultz, Aedan Pope, Michael B. Chang, Cheng Li, Laurent El Shafey, Michela Paganini, Sholto Douglas, Bernd Bohnet, Fabio Pardo, Seth Odoom, Mihaela Rosca, Cicero Nogueira dos Santos, Kedar Soparkar, Arthur Guez, Tom Hudson, Steven Hansen, Chulayuth Asawaroengchai, Ravi Addanki, Tianhe Yu, Wojciech Stokowiec, Mina Khan, Justin Gilmer, Jaehoon Lee, Carrie Grimes Bostock, Keran Rong, Jonathan Caton, Pedram Pejman, Filip Pavetic, Geoff Brown, Vivek Sharma, Mario Lučić, Rajkumar Samuel, Josip Djolonga, Amol Mandhane, Lars Lowe Sjösund, Elena Buchatskaya, Elspeth White, Natalie Clay, Jiepu Jiang, Hyeontaek Lim, Ross Hemsley, Zeyncep Cankara, Jane Labanowski, Nicola De Cao, David Steiner, Sayed Hadi Hashemi, Jacob Austin, Anita Gergely, Tim Blyth, Joe Stanton, Kaushik

Shivakumar, Aditya Siddhant, Anders Andreassen, Carlos Araya, Nikhil Sethi, Rakesh Shivanna, Steven Hand, Ankur Bapna, Ali Khodaei, Antoine Miech, Garrett Tanzer, Andy Swing, Shantanu Thakoor, Lora Aroyo, Zhufeng Pan, Zachary Nado, Jakub Sygnowski, Stephanie Winkler, Dian Yu, Mohammad Saleh, Loren Maggiore, Yamini Bansal, Xavier Garcia, Mehran Kazemi, Piyush Patil, Ishita Dasgupta, Iain Barr, Minh Giang, Thais Kagohara, Ivo Danihelka, Amit Marathe, Vladimir Feinberg, Mohamed Elhawaty, Nimesh Ghelani, Dan Horgan, Helen Miller, Lexi Walker, Richard Tanburn, Mukarram Tariq, Disha Shrivastava, Fei Xia, Qingze Wang, Chung-Cheng Chiu, Zoe Ashwood, Khuslen Baatarsukh, Sina Samangooei, Raphaël Lopez Kaufman, Fred Alcober, Axel Stjerngren, Paul Komarek, Katerina Tsihlas, Anudhyan Boral, Ramona Comanescu, Jeremy Chen, Ruibo Liu, Chris Welty, Dawn Bloxwich, Charlie Chen, Yanhua Sun, Fangxiaoyu Feng, Matthew Mauger, Xerxes Dotiwalla, Vincent Hellendoorn, Michael Sharman, Ivy Zheng, Krishna Haridasan, Gabe Barth-Maron, Craig Swanson, Dominika Rogozińska, Alek Andreev, Paul Kishan Rubenstein, Ruoxin Sang, Dan Hurt, Gamaleldin Elsayed, Renshen Wang, Dave Lacey, Anastasija Ilić, Yao Zhao, Adam Iwanicki, Alejandro Lince, Alexander Chen, Christina Lyu, Carl Lebsack, Jordan Griffith, Meenu Gaba, Paramjit Sandhu, Phil Chen, Anna Koop, Ravi Rajwar, Soheil Hassas Yeganeh, Solomon Chang, Rui Zhu, Soroush Radpour, Elnaz Davoodi, Ving Ian Lei, Yang Xu, Daniel Toyama, Constant Segal, Martin Wicke, Hanzhao Lin, Anna Bulanova, Adrià Puigdomènech Badia, Nemanja Rakićević, Pablo Sprechmann, Angelos Filos, Shaobo Hou, Víctor Campos, Nora Kassner, Devendra Sachan, Meire Fortunato, Chimezie Iwuanyanwu, Vitaly Nikolaev, Balaji Lakshminarayanan, Sadegh Jazayeri, Mani Varadarajan, Chetan Tekur, Doug Fritz, Misha Khalman, David Reitter, Kingshuk Dasgupta, Shourya Sarcar, Tina Ornduff, Javier Snaider, Fantine Huot, Johnson Jia, Rupert Kemp, Nejc Trdin, Anitha Vijayakumar, Lucy Kim, Christof Angermueller, Li Lao, Tianqi Liu, Haibin Zhang, David Engel, Somer Greene, Anaïs White, Jessica Austin, Lilly Taylor, Shereen Ashraf, Dangyi Liu, Maria Georgaki, Irene Cai, Yana Kulizhskaya, Sonam Goenka, Brennan Saeta, Ying Xu, Christian Frank, Dario de Cesare, Brona Robenek, Harry Richardson, Mahmoud Alnahlawi, Christopher Yew, Priya Ponnapalli, Marco Tagliasacchi, Alex Korchemniy, Yelin Kim, Dinghua Li, Bill Rosgen, Kyle Levin, Jeremy Wiesner, Praseem Banzal, Praveen Srinivasan, Hongkun Yu, Çağlar Ünlü, David Reid, Zora Tung, Daniel Finchelstein, Ravin Kumar, Andre Elisseeff, Jin Huang, Ming Zhang, Ricardo Aguilar, Mai Giménez, Jiawei Xia, Olivier Dousse, Willi Gierke, Damion Yates, Komal Jalan, Lu Li, Eri Latorre-Chimoto, Duc Dung Nguyen, Ken Durden, Praveen Kallakuri, Yaxin Liu, Matthew Johnson, Tomy Tsai, Alice Talbert, Jasmine Liu, Alexander Neitz, Chen Elkind, Marco Selvi, Mimi Jasarevic, Livio Baldini Soares, Albert Cui, Pidong Wang, Alek Wenjiao Wang, Xinyu Ye, Krystal Kallarackal, Lucia Loher, Hoi Lam, Josef Broder, Dan Holtmann-Rice, Nina Martin, Bramandia Ramadhana, Mrinal Shukla, Sujoy Basu, Abhi Mohan, Nick Fernando, Noah Fiedel, Kim Paterson, Hui Li, Ankush Garg, Jane Park, DongHyun Choi, Diane Wu, Sankalp Singh, Zhishuai Zhang, Amir Globerson, Lily Yu, John Carpenter, Félix de Chaumont Quitry, Carey Radebaugh, Chu-Cheng Lin, Alex Tudor, Prakash Shroff, Drew Garmon, Dayou Du, Neera Vats, Han Lu, Shariq Iqbal, Alex Yakubovich, Nilesh Tripuraneni, James Manyika, Haroon Qureshi, Nan Hua, Christel Ngani, Maria Abi Raad, Hannah Forbes, Jeff Stanway, Mukund Sundararajan, Victor Ungureanu, Colton Bishop, Yunjie Li, Balaji Venkatraman, Bo Li, Chloe Thornton, Salvatore Scellato, Nishesh Gupta, Yicheng Wang, Ian Tenney, Xihui Wu, Ashish Shenoy, Gabriel Carvajal, Diana Gage Wright, Ben Bariach, Zhuyun Xiao, Peter Hawkins, Sid Dalmia, Clement Farabet, Pedro Valenzuela, Quan Yuan, Ananth Agarwal, Mia Chen, Wooyeol Kim, Brice Hulse, Nandita Dukkipati, Adam Paszke, Andrew Bolt, Kiam Choo, Jennifer Beattie, Jennifer Prendki, Harsha Vashisht, Rebeca Santamaria-Fernandez, Luis C. Cobo, Jarek Wilkiewicz, David Madras, Ali Elqursh, Grant Uy, Kevin Ramirez, Matt Harvey, Tyler Liechty, Heiga Zen, Jeff Seibert, Clara Huiyi Hu, Andrey Khorlin, Maigo Le, Asaf Aharoni, Megan Li, Lily Wang, Sandeep Kumar, Norman Casagrande, Jay Hoover, Dalia El Badawy, David Soergel, Denis Vnukov, Matt Miecnikowski, Jiri Simsa, Praveen Kumar, Thibault Sellam, Daniel Vlasic, Samira Daruki, Nir Shabat, John Zhang, Guolong Su, Jiageng Zhang, Jeremiah Liu, Yi Sun, Evan Palmer, Alireza Ghaffarkhah, Xi Xiong, Victor Cotruta, Michael Fink, Lucas Dixon, Ashwin Sreevatsa, Adrian Goedeckemeyer, Alek Dimitriev, Mohsen Jafari, Remi Crocker, Nicholas FitzGerald, Aviral Kumar, Sanjay Ghemawat, Ivan Philips, Frederick Liu, Yannie Liang, Rachel Sterneck, Alena Repina, Marcus Wu, Laura Knight, Marin Georgiev, Hyo Lee, Harry Askham, Abhishek Chakladar, Annie Louis, Carl Crous, Hardie Cate, Dessie Petrova, Michael Quinn, Denese Owusu-Afriyie, Achintya Singhal, Nan Wei, Solomon Kim, Damien Vincent, Milad Nasr, Christopher A. Choquette-Choo, Reiko Tojo, Shawn Lu, Diego de Las Casas, Yuchung Cheng, Tolga Bolukbasi, Katherine Lee, Saaber Fatehi, Rajagopal Ananthanarayanan, Miteyan Patel, Charbel Kaed, Jing Li, Shreyas Rammohan Belle,

Zhe Chen, Jaclyn Konzelmann, Siim Põder, Roopal Garg, Vinod Koverkathu, Adam Brown, Chris Dyer, Rosanne Liu, Azade Nova, Jun Xu, Alanna Walton, Alicia Parrish, Mark Epstein, Sara McCarthy, Slav Petrov, Demis Hassabis, Koray Kavukcuoglu, Jeffrey Dean, and Oriol Vinyals. Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context, 2024.

Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, Dan Bikel, Lukas Blecher, Cristian Canton Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, Brian Fuller, Cynthia Gao, Vedanuj Goswami, Naman Goyal, Anthony Hartshorn, Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madian Khabsa, Isabel Kloumann, Artem Korenev, Punit Singh Koura, Marie-Anne Lachaux, Thibaut Lavril, Jenya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Molybog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein, Rashi Rungta, Kalyan Saladi, Alan Schelten, Ruan Silva, Eric Michael Smith, Ranjan Subramanian, Xiaoqing Ellen Tan, Binh Tang, Ross Taylor, Adina Williams, Jian Xiang Kuan, Puxin Xu, Zheng Yan, Iliyan Zarov, Yuchen Zhang, Angela Fan, Melanie Kambadur, Sharan Narang, Aurelien Rodriguez, Robert Stojnic, Sergey Edunov, and Thomas Scialom. Llama 2: Open foundation and fine-tuned chat models, 2023.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need, 2023.

Yuxin Wu and Kaiming He. Group normalization, 2018.

Songlin Yang, Bailin Wang, Yikang Shen, Rameswar Panda, and Yoon Kim. Gated linear attention transformers with hardware-efficient training, 2024.

A Author Contributions

Daniel Goldstein Entire GPTAlpha design, research, and code. GoldFinch code, architecture design, and research. Full manuscript initial draft except 4.3. Manuscript edits. Proofreading and revisions of full manuscript. Core experiments featured herein.

Fares Obeid Research discussions and experiments during development of the GoldFinch architecture. Significant input on all aspects of final architecture design.

Eric Alcaide Research discussions and experiments during development of the GoldFinch architecture. Significant input and experiments leading to Finch-C2 design.

Guangyu Song Section 4.3. Experiments for 4.3.

Eugene Cheah GoldFinch code proofreading, development of release code and testing, contributions to pre-fill mechanism details.

B Other Related Work

Ring Attention (Liu et al., 2023) allows the attention calculation to be split across many discrete processors that do not share VRAM. Keys and values can be split up among these processors, linearly amortizing the amount of KV-Cache required to remain resident within each processor's VRAM. This enables unbounded scaling of attention given enough hardware, but does not address the cost of $O(N^2)$ compute, and still imposes total memory costs that scale with the sequence length.